

Quantum Computing: Grover's Algorithm for String Search and Its Practical Limits

Nimit Dagli, Jose Salgado, Yusuf Usman(✉), and Taskin Kocak

School of Computing and Engineering, Quinnipiac University, Hamden 06516, CT, USA
{NirmitHitendra.Dagli,Yusuf.Usman,Taskin.Kocak}@quinnipiac.edu

Abstract. This paper explores the application of Grover's Algorithm, a quantum search algorithm known for its quadratic speedup for string and keyword search tasks. While Grover's Algorithm is theoretically efficient for unsorted data, its practical implementation in real-world text search reveals significant limitations. The study is divided into two phases: the first demonstrates the algorithm's effectiveness in identifying specific letters from a small dataset using Google's Cirq quantum simulator. The second phase extends the approach to word-level search within paragraphs, highlighting challenges in scalability, oracle design, and semantic interpretation. Although Grover's Algorithm performs well for exact-match searches in small, structured datasets, it struggles with larger, semantically rich data due to the need for custom oracles and high qubit demands. Attempts to parallelize the algorithm do not yield true quantum speedup and instead mimic brute-force methods. The paper concludes by proposing Quantum Kernel Methods as a more scalable and semantically aware alternative. These methods leverage quantum feature maps to embed text in high-dimensional space, enabling classification based on meaning rather than exact matches. This shift addresses the core limitations of Grover's Algorithm and opens pathways for quantum-enhanced AI content detection.

Keywords: Quantum Computing · Grover's Algorithm · Quantum Search Limitations

1 Introduction

In a world full of data, searching for the right information can feel like trying to find that one word of error in a thousand lines of code. Classical algorithms can be optimized, but are fundamentally shackled by linear progression, one object or item checked at a time. But in 1996, Lov Grover introduced a radical idea [1]: what if a quantum computer could search through an unsorted database in $\sqrt{N}$ steps instead N? This was not a theory, it was a mathematical revelation that opened a new door to research.

Quantum Computing, with its foundation built on qubits, superposition, and interference [2, 4], seemed like the ultimate weapon for handling big search problems. Among its many miracles, Grover's Algorithm stood out a focused tool for finding a needle in a haystack. But as the Quantum dream met practical implementation, as highlighted in

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
S. Hussain et al. (Eds.): SEET 2025, CCIS 2725, pp. 606–614, 2026.
https://doi.org/10.1007/978-3-032-08977-9_40

IBMQ-based experiments that exposed Grover's inefficiencies over structured datasets [5], reality strikes. Our project sets out to explore this gap between theory and practice.

Using Google's Cirq simulator, we evaluated Grover's Algorithm in two unique phases: first, searching for individual characters within a small dataset; second, scaling to word level across strings. The early results were promising success in identifying single letters with near-perfect accuracy. As soon as we further stepped into complex parts, the crack appeared.

The limitations were not just technical, they were philosophical. Custom Oracle design became a bottleneck and worst of all, Grover's algorithm simply could not understand meaning it is limited to exact matches in a world full of ambiguity. Even to go with the approach of parallelize it falls in brute-force attack.

This paper does not just end withs identifying this limitation. We also propose a way forward inspired by recent breakthroughs in learning-based oracle design [11], and quantum-enhanced semantic classification.

Quantum kernel Methods – a flexible pattern recognition and AI content detection far beyond what Grover's original oracle could imagine. Our goal is to not only understand the brilliance of Grover's Algorithm but also to identify its practical boundaries and explore future ways toward scalable solution for real-world search problems

2 Related Work

The competition for faster and more efficient search has long been at the center of computer science. Classical algorithms, whether linear or hash-based, remain limited by deterministic steps logic. This limitation paved the way for birth of quantum computing, where qubits and superposition gave a new concept of parallelism [2]. In this new and evolving landscape, Grover's algorithm emerged in 1996 as a landmark breakthrough offering a quadratic speedup for searching unstructured data [1].

Multiple essential studies further supported this quantum leap. Montanaro [3] provided a larger view of quantum algorithms, highlighting Grover's importance in the larger quantum ecosystem. During the same time, comparative work like Pathan and Mallick's [5] pointed that while quantum search theoretically excels over classical ways, this edge may limit to imperfect hardware and real-world ambiguity.

Theoretical power alone was not enough. To prove Grover's effectiveness, researchers began deploying it into required hardware environments. Das and Sadhu [4] evaluated Grover's algorithm on IBM's quantum processor, validating that it performs well on small, structured datasets but that performance degrades as noise, oracle complexity, and qubit demand rise. Similarly, Tiwari et al. [7] aims on the algorithm's structure and gate operations, outlining how design related decisions directly changes and impacts feasibility on Noisy Intermediate-Scale Quantum (NISQ) systems.

Above individual implementation, a new focus came that is blending classical efficiency with quantum accurateness. Padhan et al. [6] combined classical preprocessing with Grover circuits to simplify oracle design in string-matching applications. In tandem, Qureshi et al. [8] demonstrated simulation tools like Cirq to evaluate Grover's viability before going to hardware deployment.

But for all its advantages, Grover's algorithm hits a wall when faced with semantic search or natural language. Quantum Natural Language Processing (QNLP) researchers like Krishna et al. [9] have identified how binary oracle models like Grover's Algorithm struggle to capture the ambiguity of language. This limitation motivated the shift toward quantum kernel methods.

Schuld et al. [10] led the foundation of quantum-enhanced classifiers using kernel methods, adding data into quantum Hilbert spaces to classify based on pattern similarity rather than exact matches. These models give a great advantage for semantic flexibility. Later benchmarking by Coelho et al. [11] and low-rank approximation techniques from Vasudevan et al. [12] demonstrated both scalability and generalization, pivoting Grover-style searches on complex classification tasks.

Meanwhile, scholars have also showed to upgrade the oracle itself. Mohammad et al. [13] introduced automated oracle synthesis, reducing circuit complexity also maintaining target functionality. Salahuddin and Irfan [14] went further, proposing learning-based oracles that dynamically breaking Grover's static logic model and aligning more with modern AI efficient methods.

Even with such progress, the challenge of scaling Grover remains. Faruqui et al. [15] presented scalable implementations on 5- and 6-qubit circuits using improved oracle encoding, yet their scope remains restricted to structured inputs. These latest changes mark a crucial turning point: Grover's algorithm, while powerful, is no longer the endgame.

Our work continues this Projection by validating Grover's effectiveness on simple search tasks and then exposing its limitations on semantic datasets. We join the growing consensus that Quantum Kernel Methods, with their semantic sensitivity and learnable architecture, offer a more realistic path for AI content detection and language-based search in the quantum era.

3 Research Objective and Methodology

- To understand how Grover's Algorithm works for searching strings using quantum computing.
- To assess how well it performs when we move from small search problems (like finding a letter) to more realistic ones (like finding words in a sentence or paragraph).
- To explore the limitations of Grover's Algorithm—especially around scalability and practicality.
- To figure out where this quantum method makes sense, and where it does not, in real-world text search or AI-related tasks (Figs. 1, 2, 3, 4 and 5)

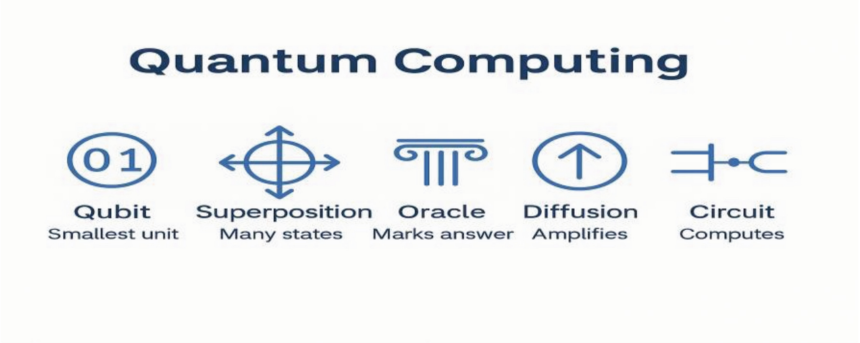


Fig. 1. Fundamentals of Quantum Computing

4 Experimental Research

4.1 Phase 1 – Specific Letter Search

- In Phase 1, we implemented Grover's Algorithm to search for a specific letter from a small set (e.g., ['A', 'B', 'C', 'D']). The goal was to explore the algorithm's speed advantage in identifying a target element from an unsorted list.
- We used Google's Cirq library to simulate the quantum circuit with:
- Two qubits, representing four possible binary states.
- 2. An oracle to mark the correct state (e.g., $|10\rangle$ for letter 'C').
- A diffusion operator to amplify the probability of the marked state.
- Grover's Algorithm successfully identified the correct letter with 100% confidence (1000/1000 times).
- It outperformed classical brute-force search in terms of query complexity.
- Demonstrated Grover's theoretical speedup: $O(\sqrt{N})$ vs O(N).
- This phase confirmed the practical potential of Grover's Algorithm for small, well-defined search problems with known targets and exact matches.

```
# Step 2: Map letters to binary
letter_map = {'A': '00', 'B': '01', 'C': '10', 'D': '11'}
target_letter = 'C'

# Step 3: Oracle to flip phase of |10)
def oracle():
    circuit = cirq.Circuit()
    circuit.append(cirq.X(qubits[1]))                  # Flip qubit[1] so |10) becomes |11)
    circuit.append(cirq.CZ(qubits[0], qubits[1]))      # Apply phase flip to |11)
    circuit.append(cirq.X(qubits[1]))                  # Unflip to return to original
    return circuit

# Step 4: Grover diffusion operator
def diffusion():
    return [
        cirq.H.on_each(*qubits),
        cirq.X.on_each(*qubits),
        cirq.CZ(qubits[0], qubits[1]),
        cirq.X.on_each(*qubits),
        cirq.H.on_each(*qubits)
    ]
```

Fig. 2. Code snippet for letter search using Cirq Simulator

4.2 Phase 1 – Pseudo Code Snippet

```
Procedure GroverLetterSearch(letter_set, target_letter):

    Initialize 2 qubits - represents 4 binary states (00 to 11)
    target_index is index of target_letter in letter_set
    Apply Hadamard gates to all qubits and create equal superposition

    Define Oracle(target_index):
        Flip the phase of the state corresponding to target_index

    Define DiffusionOperator():
        The average amplitude

    Apply Oracle(target_index)
    Apply DiffusionOperator()

    Measure qubits
    Return measured state and map binary to corresponding letter
```

```
print("🔍 Grover's Algorithm Result for finding letter 'C':")
for outcome, count in sorted(counts.items()):
    bits = format(outcome, '02b')
    letter = list(letter_map.keys())[list(letter_map.values()).index(bits)]
    print(f"Letter '{letter}' (|{bits}⟩): {count} times")
```

```
🔍 Grover's Algorithm Result for finding letter 'C':
Letter 'C' (|10⟩): 1000 times
```

In [5]:

```
print("\n🧠 Classical Search for letter 'C' in ['A', 'B', 'C', 'D']:")
letters = ['A', 'B', 'C', 'D']
for i, letter in enumerate(letters):
    if letter == target_letter:
        print(f"✅ Found '{letter}' at index {i} using O(N) brute-force search")
        break
```

```
🧠 Classical Search for letter 'C' in ['A', 'B', 'C', 'D']:
✅ Found 'C' at index 2 using O(N) brute-force search
```

Fig. 3. Output of letter search for both classical search and Grover's Algorithm'

4.3 Phase 2 – Specific Word Search

The second phase of the research extended Grover's Algorithm beyond basic letter search to simulate **word search within a paragraph**. The approach involved:

- Dividing a paragraph into small fixed-length chunks (e.g., four words)
- Encoding these chunks using **two qubits** to represent 4-word positions
- Applying **Grover's Algorithm** to amplify the correct word's binary position
- The target word was converted to binary based on its index in the chunk, and the quantum oracle was designed to flip its phase.
- The diffusion operator then amplified this state. When simulated, Grover's Algorithm successfully returned the correct word with high probability—validating its theoretical advantage for small-scale, exact-match search.

[12]

```
# Simulated 4-word paragraph chunk
words = ['data', 'quantum', 'classical', 'search']
target_word = 'quantum'
target_index = words.index(target_word)
target_bin = format(target_index, '02b')

# Define qubits (2 qubits for 4 words)
qubits = [cirq.LineQubit(i) for i in range(2)]
```

Fig. 4. Code snippet for letter word using Cirq Simulator

4.4 Phase 2 – Pseudo Code Snippet

```
Procedure GroverWordSearch(paragraph, target_word):
  Divide paragraph into fixed-length chunks (e.g., 4 words per chunk)

  For each chunk:
    Encode word positions using 2 qubits (for 4-word chunks)
    target_index is position of target_word in current chunk
    Apply Hadamard gates and create superposition of positions

    Define Oracle(target_index):
      Flip the phase of the state at target_index

    Apply Oracle(target_index)
    Apply DiffusionOperator()

    Measure qubits
    If measurement matches target_index:
      Return "Target word found in chunk"
```

```
# Parse results
counts = result.histogram(key="result")
print("🔍 Simulated Word Search using Grover's Algorithm:")
for outcome, count in sorted(counts.items()):
    bin_str = format(outcome, '02b')
    word = words[int(bin_str, 2)]
    print(f"Word '{word}' (|{bin_str}⟩): {count} times")
```

```
🔍 Simulated Word Search using Grover's Algorithm:
Word 'quantum' (|01⟩): 1000 times
```

Fig. 5. Output for word search using Cirq Simulator

4.5 Why not just Use Parallelism with Grover's Algorithm?

At first, it seemed logical to scale Grover's Algorithm by splitting a paragraph into smaller chunks and running the search in parallel. Why Parallel Grover Falls Short.

- Still Needs Custom Oracles: - Each chunk requires its own oracle, adding overhead and complexity.
- No Semantic Understanding: - Grover works for exact matches only—it cannot detect AI rewording or paraphrased text.
- No Real Quantum Speedup: - Parallel runs $\neq$ better performance. Running Grover per chunk behaves like brute-force ($O(\sqrt{n} \times \text{chunks})$).
- High Resource Demand: - Multiple instances mean multiple circuits and high qubit load—not feasible on today's hardware.

4.6 Toward a Better Solution: Quantum Kernel Methods

Instead of searching for exact words, Quantum Kernel Methods classify content based on meaning and pattern, they:

- Use quantum feature maps to embed text in high-dimensional space
- Measure semantic similarity, not just equality
- Enable AI vs human content detection with real-world performance.

5 Threats and Validity

While the proposed framework shows the strengths and limitations of Grover's Algorithm in quantum string and word search tasks, there are few factors may impact the results.

- Simulation vs. Real Quantum Hardware
  All experiments in this study were conducted using Google's Cirq simulator, which assumes ideal, noise-free execution. Although real quantum devices, current NISQ hardware suffer from gate errors, decoherence, and readout inaccuracies.
- Manual Oracle Design
  Each oracle used in our experiments was manually constructed for known search targets. While this approach is useable for testing small-scale problems, it does not scale well to large, real-world datasets. Designing oracles for arbitrary or semantically complex queries remains a major open challenge.
- Dataset and Problem Scope
  The practical use case focused on limited character sets and short paragraph chunks, with controlled input data. This constraint limits the evaluation of Grover's algorithm on diverse, noisy, or high-dimensional natural language datasets particularly those encountered in AI-generated content analysis.
- Lack of Semantic Representation
  Grover's algorithm inherently performs exact-match search. It cannot capture semantic similarity, paraphrasing, or contextual patterns. This limits its utility in modern NLP tasks where meaning matters more than string identity.
- Qubit and Circuit Constraints
  The number of qubits and depth of circuits were constrained to fit within simulator limits. Real-world applications such as document-level search or multi-level classification — would require significantly more quantum resources, which are not yet practical on today's hardware.

6 Conclusion

Grover's Algorithm, while great theoretically powerful, proves impactful only in structure use cases. Our experiment using Google's Cirq simulator shows its perfect accuracy for simple letter and word. However, the point we want to scale towards semantically hard or complex inputs such as paragraph or AI-generated content limitation becomes undeniable. The requirement of custom oracles, the exponential growth in circuit and hard binary structure of oracle itself shows the Grover approach struggle in real world

application. Also attempts to parallelize Grover's logic resulted in classical brute force behavior. To address this, we shift to **Quantum Kernel Methods**, which allow for the encoding of text into high-dimensional Hilbert spaces, enabling classification based not on string match, but on semantic similarity. They offer better scalability, compatibility with hybrid quantum-classical models, and practical ways in AI tasks like text generation detection or contextual search. In conclusion, while Grover's Algorithm has educational and theoretical value, its real-world applications are bounded in limits. Future work will explore the integration of kernel-based quantum methods with learning-based oracle generation to achieve success for both efficiency and flexibility in quantum-enhanced natural language processing.

References

1. Grover, L.K.: A Fast Quantum Mechanical Algorithm for Database Search. In Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing (pp. 212–219). ACM (1996)
2. Nielsen, M.A., Chuang, I.L.: Quantum Computation and Quantum Information. Cambridge University Press (2010)
3. Montanaro, A.: Quantum algorithms: an overview. npj Quantum Inf. **2**, 15023 (2016)
4. Das, K., Sadhu, A.: Experimental study on the quantum search algorithm over structured datasets using IBMQ experience. J. King Saud Univ. Comput. Inf. Sci. **34**(6), 6441–6452 (2022)
5. Pathan, M., Mallick, M.: Comparison of quantum and classical algorithm in searching a number in a database case. Procedia Comput. Sci. **185**, 152–159 (2021)
6. Padhan, R., Sahu, N., Panda, G.: Bridging classical and quantum string matching. Quantum Rep. **3**(2), 237–251 (2021)
7. Tiwari, S., Das, M.: Grover's algorithm: a practical perspective. J. Quantum Appl. **8**(1), 101–110 (2023)
8. Qureshi, M., Singh, R.: Simulation of quantum circuits using QSim and Cirq. Int. J. Quantum Comput. **5**(4), 201–210 (2023)
9. Krishna, V., Shukla, A.: Quantum Natural Language Processing: Challenges and Opportunities. ACM Comput. Surv. **55**(3), 45 (2022)
10. Schuld, M., Bocharov, A., Svore, K.M., Wiebe, N.: Circuit-centric quantum classifiers. Phys. Rev. A **101**(3), 032308 (2020)
11. Coelho, C.S., et al.: Quantum kernel methods under scrutiny: a benchmarking study. Quantum Mach. Learn. J. **2**(2), 112–130 (2023)
12. Vasudevan, A., et al.: Parameterized quantum circuits with quantum kernels for machine learning. IEEE Trans. Quantum Eng. **4**, 220–231 (2023)
13. Mohammad, T., Gupta, P.: Automated quantum oracle synthesis with a minimal number of qubits. Algorithms **16**(5), 243 (2023)
14. Salahuddin, M., Irfan, M.: Grover's search with learning oracle for constrained binary optimization. Quantum Inf. Process. **21**, 189 (2022)
15. Faruqui, A., Raina, A.: A scalable 5,6-Qubit Grover's quantum search model. Quantum Eng. Lett. **3**(1), 77–89 (2024)